COURSE-BASED PROJECT DOCUMENTATION | DOC-03

Technology Stack & Toolchain

Tools, Exact Documented Versions, & Architectural Roles

1. Technology Stack Overview

The project uses an industry-standard, multi-language systems toolchain consisting of C99, GNU Flex, GNU Bison, and Python 3.10+. Each tool occupies a strictly defined layer in the compilation and presentation stack.

Technology / Tool	Exact Documented Version	Role in Project Architecture
Target Language	Java 21 (LTS)	Language specification targeted by grammar r...
GNU Flex	2.6.x	Generates lexical scanner (lex.yy.c) from re...
GNU Bison	3.8.x	Generates LALR(1) shift-reduce parser (java_...
GCC Compiler	MinGW-w64 (MSYS2 UCRT64)	Compiles C sources into javacheck.dll and ja...
Python	3.10+	Runs desktop GUI and orchestrates automated ...
Tkinter	Built-in Standard Library	GUI toolkit rendering editor, line numbers, ...
ctypes	Built-in Standard Library	In-memory C Foreign Function Interface loadi...
Build Scripts	CMD, PowerShell, Make	Automated multi-platform compilation pipelines
Test Runner	Python unittest	Automated regression testing of parser and GUI

2. Detailed Role of Every Component

GNU Flex (Lexical Analyzer)

Compiles regular expressions into an optimized Deterministic Finite Automaton (DFA). Eliminates whitespace and comments while emitting discrete token integers and tracking exact line coordinates.

GNU Bison (Syntax Parser)

Translates Context-Free Grammar rules into a deterministic LALR(1) state machine. Manages operator precedence, resolves ambiguities, and catches syntax violations via yyerror().

GCC / MinGW-w64 UCRT64 (Native Compiler)

Compiles the generated C files into a high-performance shared library (javacheck.dll) exporting check_syntax() and a standalone CLI binary (javacheck.exe).

Python / Tkinter / ctypes (User Interface & FFI)

Provides zero-dependency, cross-platform presentation. Uses ctypes to invoke the native DLL directly in memory without process spawning overhead.